GPG (GNU Privacy Guard): Management and Use of Asymmetric Keys

NECKER

November 18, 2025

Contents

1 GPG: Foundations and Web of Trust

This section introduces GPG (GNU Privacy Guard), its context, and the concepts of Trust and Validity that distinguish it from traditional Public Key Infrastructure (PKI).

1.1 PKI vs. GPG

GPG (GNU (GNU's Not Unix: hence a free open-source project) Privacy Guard) is the software suite that replaces the **PGP (Pretty Good Privacy**, proprietary software but with the same functionality as its successor **GPG**) protocol and is fully compliant with the **OpenPGP** standard. Although both utilize asymmetric cryptography, they differ drastically in their trust management model.

Table 1: Trust Models Comparison: PKI vs. PGP/GPG

Feature	**PKI (Example: HTTPS)**	**PGP/GPG (Web of Trust)**
Source of Trust	**Hierarchical/Centralized** (Certificate Authority - CA).	**Decentralized** (Individual users).
Basis of Trust	Trust in X's key is guaranteed by the **signature of an accredited** and pre-approved CA.	Trust in X's key derives from the **signatures of other users** who trust X.
Identity Verification	**CA Duties** (verifies the identity of the certificate owner).	**User Duties** (each user verifies the identity and signs the keys they trust).

1.2 Concept of Trust (*Trust*) in PGP/GPG

In GPG, **trust** (*Trust*) reflects how much a user trusts another user in the accuracy and reliability of **verifying third-party identities** and signing their keys.

- **Owner Trust:** A list for each key containing all entities to whom I assign a degree of trust in verifying it (this value must be **manually set by the user**) for each key present in their own portachiavi (*keyring*).
- **Purpose:** To determine who is authorized to sign keys in one's own *keyring* to validate them.
- **Trust Levels:** Trust can be set to different levels:
  - **Full Trust:** The user fully trusts that this person correctly verifies and signs the keys of others.
  - **Marginal Trust:** The user partially trusts or trusts with reservations this person in verifying others' keys.
  - **Untrusted:** The user does not trust them at all.

1.3 Key Validity (*Key Validity*)

Key validity is the certainty that the public key in question actually belongs to the entity with whom one wishes to communicate. It is a value automatically calculated by GPG based on two factors:

$$\text{Validity} = f(\text{Owner Trust}, \text{Number of Signatures on the Key})$$

Validity determines whether it is safe to encrypt data for that key.

- **Validity and Signatures:** For a key to be considered "**valid**" by GPG in one's system, the following are required:
  - **A single signature** from a key that the user has marked with **Full Trust**,
  - **Or** at least **three signatures** from keys that the user has marked with **Marginal Trust**.
- **Flow:** The user sets the *Trust* → PGP/GPG computes the *Validity* based on that Trust and the number of signatures.

1.4 Symmetric Cryptography with GPG (Without Asymmetric Keys)

In addition to asymmetric use, GPG can be used solely for symmetric cryptography, based on a passphrase (symmetric key to be used for encryption).
To encrypt a file, simply open a terminal and type:

Table 2: GPG Symmetric Cryptography

Command	**Description**
`gpg --symmetric <input>`	Encrypts the specified file using a symmetric algorithm (e.g., AES) protected by a **passphrase**. The public key is not required.
`[--cipher-algo <algo>]`	Specifies the symmetric algorithm to use (e.g., `AES256`).
`[--output <name>]`	Specifies the name of the output encrypted file.
`[--armor]`	Uses the ASCII Armored format (`.asc`, ensures no data corruption during transport).
`gpg -d <data>`	Decrypts symmetrically encrypted data. Requires the correct **passphrase**.

```
gpg [--output <encrypted_file_name>] --symmetric [--cipher-algo <algorithm>] [--armor] <
    input_file>

Example:
gpg --output document.txt.gpg --symmetric document.txt




gpg [--output <decrypted_file_name>] -d <encrypted_file>

Example:
gpg --output original_document.txt -d document.txt.gpg
```

2 Generation and Management of Asymmetric Keys

The `gpg` commands for generating and managing keys are fundamental to begin using asymmetric cryptography.

2.1 Generation of Asymmetric Keys

GPG key generation creates a key pair: a **public key** (to be distributed) and a **secret/private key** (to be kept secure).

Table 3: Commands for GPG Key Generation

`Command`	**Mode**	**Description**
`gpg --generate-key` `gpg --gen-key`	Fast	Rapid generation. Requires only name, email, and passphrase. Default options (algorithm, length, expiration) are used automatically.
`gpg --full-generate-key` `gpg --full-gen-key`	Full	Complete generation. Allows detailed specification of the algorithm (e.g., RSA, ECC), key length (e.g., 2048, 4096 bits), and expiration date.

2.2 Listing and Inspection of Keys

To view and manage the keys present in one's own `keyring`.

Table 4: Commands for Listing and Inspecting Keys

`Command`	**Description**
`gpg --list-keys` `gpg -k`	Lists all **public keys** present in the user's `keyring`. The **Key ID** and **Fingerprint** (hashed public key) are shown.
`gpg --list-secret-keys`	Lists all **secret (private) keys** present in the user's `keyring`. These keys are required to decrypt and sign.
`gpg --edit-key <key>`	Opens the interactive modification interface for the specified key (via UID, Key ID, or Fingerprint). Shows additional information such as the **trust** level and allows advanced operations such as: • Adding new `UIDs` (User IDs). • Adding `subkeys`. • Setting the trust level (`trust`). • Signing other keys.

3 Data Encryption and Signing

These are the main commands to apply the basic functions of asymmetric cryptography: confidentiality and authentication.

3.1 Asymmetric Encryption (Confidentiality)

To encrypt data, the recipient's **public key** is required. Only the recipient can decrypt it with their **private key**.

Table 5: Data Encryption and Decryption

Syntax and Options	**Function**
`gpg --encrypt <data>` `gpg -e <data>`	Encrypts the specified data (usually a file).
`-r <recipient>`	Specifies the identifier of the recipient. Their public key will be used for encryption. If omitted, GPG will prompt for recipients.
`[--output <name>]`	Specifies the name of the output encrypted file (default: appends `.gpg` or `.asc`).
`[--armor]`	Output in ASCII Armored format (`.asc`).
`gpg -d <data>`	Decrypts the specified data. Requires the recipient's **secret key** in the local `keyring` and the `passphrase`.
`[--local-user <UID>]`	Specifies which secret key to use for decryption, useful if the user owns multiple keys.

```
gpg -e -r <Recipient> [--armor] <input_file>

Example:
gpg --encrypt --recipient alice --armor report.pdf



gpg [--output <decrypted_file_name>] -d <encrypted_file>

Example:
gpg --output original_document.txt -d document.txt.gpg
```

3.2 Data Signing (Authentication and Integrity)

Signing uses the sender's **secret key** to authenticate the origin and ensure that the data has not been tampered with.

Table 6: Commands for Data Signing

Command and Options	Description
`gpg --sign <data>`	Creates an encrypted and signed file. The signature is included within the output file.
`gpg --clearsign <data>`	Creates a readable output (not encrypted, the signature is within the text body). **No compression occurs**, making the result directly readable. (without the signature, it must be added separately)
`gpg --detach-sign <data>`	Creates the signature in a **separate file** (usually `.sig` or `.asc`). The original data file remains unaltered. Useful for large files.
`[-u <UID/Fingerprint>]`	Specifies which secret key (`--local-user` in its long form) to use for signing.
`[--armor]`	Output of the signature in ASCII Armored format.
`gpg --verify <signature>`	Verifies a signature (separated or included) against the data file. Requires the sender's **public key** in the local `keyring`.

4 Key Signing (Web of Trust)

The concept of the **Web of Trust** (Network of Trust) in PGP/GPG is based on mutual key signing to establish validity.

Table 7: Key Signing and Verification

Command	Description
`gpg --sign-key <KeyId>`	Signs the specified public key (of another user) using one's own secret key. By this act, one attests that the identity associated with the key has been verified.
`[-u <UID/Fingerprint>]`	Specifies which of one's own secret keys to use to sign the target key.
`gpg --list-signs <key>`	Shows the list of all keys present in the `keyring` that have signed the specific target key. This helps assess the level of trust and the propagation of the key.

5 Interaction with Keyservers and Other Users

These comands allow key exchange between users and interaction with public `keyservers`.

5.1 Importing and Exporting Keys

Table 8: Commands for Importing and Exporting Keys

Syntax and Options	**Function**
`gpg --export <UID/Fingerprint>`	Exports the specified **public key**. By default, the output is in binary format and is written to standard output.
`[--output <keyname>]`	Specifies a destination file for the exported key.
`[--armor]`	Uses the ASCII format (`.asc`) instead of binary. This readable and easy to copy/paste format is called **ASCII Armored**.
`gpg --import <filename>`	Imports the public key contained in the specified file into the user's `keyring`.

5.2 Interaction with a Key Server

Keyservers (Key Servers) act as public and decentralized registries of the Web of Trust. It is crucial to understand that they must contain only public and authenticated information, never confidential data.

- **What to Ingest (Public Data):**

  The Keyserver is intended to receive data that contributes to the **validation** and **reachability** of the key.

  1. **Your Public Key:**
     - It is the essential data that allows others to encrypt messages for you.
     - It includes your User ID (`UID`) and the **self-certification signature** generated with your private key, which attests to its initial authenticity.
  2. **The Signatures You Have Affixed (Acts of Trust):**
     - After verifying another person's identity (e.g., Mario), the **signatures on his key generated by your private key** are uploaded.
     - This notifies the world that you (the identity behind your key) trust Mario's identity, contributing to the computation of his key's validity for those who trust you.

- **Verifying Key Authenticity:**
  To verify whether a key is on a **keyserver**, I look into my **keyring** (list of people I trust) to see if anyone has signed it.

Commands to interact with the keyserver:

Table 9: Interaction with Key Servers

Command	**Description**
`gpg --keyserver <url> --search-keys <UID/KeyId>`	Searches the specified `keyserver` for the key corresponding to the provided identifier (UID, Key ID, or Fingerprint).
`gpg --keyserver <url> --send-keys <KeyId>`	Sends one's own public key (identified by the Key ID) to the specified `keyserver`, making it available to other users.
`gpg --keyserver <url> --recv-keys <KeyId>`	Searches for the specified key on the `keyserver` and imports it into the user's local `keyring`.